

Process Scheduling

Which process is given control of the CPU and for how long?

Process Scheduling

- It is the activity of managing the multiple processes.
- It is done by the **process manager**.
- It is the task of selection of next process to be executed when the running process is removed/completed.
- It is an essential part of multiprogramming OS. It allows more than one process to be loaded into the memory at a time and OS has to schedule the processes for execution.

When to Schedule

1. When a new process is created.
2. When a process terminates.
3. When a process blocks on I/O, on semaphore, waiting for child termination etc.
4. When an I/O interrupt occurs.
5. When quantum expires.

Goals of Scheduling

- **Fairness:** Each process gets fair share of the CPU.
- **Efficiency:** When CPU is 100% busy then efficiency is increased.
- **Response Time:** Minimize the response time for interactive user.
- **Throughput:** Minimizes jobs per given time period.
- **Waiting Time:** Minimizes total time spent waiting in the ready queue.
- **Turnaround Time:** Minimizes the time between submission and termination.

Process Scheduling Queues

- The OS maintains all PCBs in Process Scheduling Queues. It maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of the process is changed, its PCB is unlinked from its current queue and moved to its new state queue.
- The OS maintains the following important process scheduling queues:
 - **Job queue:** This queue keeps all the processes in the system.
 - **Ready queue:** This queue keeps a set of all processes residing in main memory, ready and waiting to be executed. A new process is always put in this queue.
 - **Device queue:** The processes which are blocked due to unavailability of an I/O device are placed in this queue.

Scheduler

- It is a program within OS that handles process scheduling.
- It's main task is to select the jobs to be submitted into the system and to decide which process to run.
- Types:
 - **Long Term Scheduler:** also called job scheduler. It determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution.
 - **Short Term Scheduler:** also called CPU scheduler. It selects a process among the processes that are ready to execute and allocates CPU to one of them.
 - **Medium Term Scheduler:** It determines the process to be removed from the memory. It is in-charge of handling the swapped-out processes.

Dispatcher

- It is a special program within OS that is executed after scheduler.
- When a scheduler completes its job of selecting a process, then the dispatcher takes the process to the desired state/queue.
- The functions of dispatcher includes:
 - Context Switching
 - Switching to User Mode
 - Jumping to proper location in user program to restart the program.

Difference between Dispatcher and Scheduler

Dispatcher

- It is a module in OS that gives control of CPU to the process selected by short term scheduler.
- Working of dispatcher is dependent on scheduler.
- It has fixed working process with no multiple algorithms for its implementation.

Scheduler

- It is a module in OS which selects a process among various processes.
- It works independently. It works immediately when required.
- It works on various algorithm such as FCFS, SJF, SRTN, RR etc.

Context Switching

- It is the process of switching the processing of one process to another process. It is done by the dispatcher.
- In multitasking OS, context switching enables multiple processes to share a single CPU.
- When the dispatcher switches the CPU from executing one process to another, the state from the current running process is stored into its PCB. After this, the state of the next process to be executed is loaded from its own PCB and execution begins. This switching is termed as Context Switching.
- It is an expensive task as the CPU time is wasted during switching.
- During context switching, the following information is stored for later use: Program Counter, Scheduling Information, Currently used register, Changed State, I/O State Information, Accounting Information.

Types of Scheduling

- Preemptive Scheduling
- Non-Preemptive Scheduling

Preemptive Scheduling

- The resources are allocated to a process for a limited time.
- Process can be interrupted in between.
- If a high priority process frequently arrives in the ready queue, low priority process may starve.
- Preemptive scheduling has the overheads of scheduling the processes.
- Preemptive scheduling is flexible.

Non-Preemptive Scheduling

- Once resource are allocated to a process, the process holds it till it completes its burst time or switches to waiting state.
- Process cannot be interrupted till it terminates or switches to waiting state.
- If a process with long burst time is running CPU, then another process with less CPU burst time may starve.
- Non-preemptive scheduling does not have overheads of scheduling the process.
- It is not flexible as preemptive.

Task

- Differentiate between preemptive and non-preemptive scheduling.

Scheduling Criteria

- Different CPU Scheduling algorithms have different properties.
- Multiple criteria have been suggested for comparing CPU scheduling algorithms. Some of them are:
 - CPU utilization: keep the CPU as busy as possible.
 - Throughput: number of processes that complete their execution per time unit.
 - Turnaround time: amount of time to execute a particular process.
 - Waiting time: amount of time a process has been waiting in the ready queue.
 - Response time: amount of time it takes from when a request was submitted until the first response, not output.

Scheduling Algorithm

- FCFS (First Come First Served)
- SJF (Shortest Job First)
- SRTN (Shortest Remaining Time Next)
- RR (Round Robin)
- Priority Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

Scheduling Algorithm

Scheduling algorithms are mainly divided into following three categories:

- Batch system scheduling
- Interactive system scheduling (online)
- Real Time scheduling

Batch System Scheduling

- Batch processing is a technique in which an OS collects the programs and data together in a batch before processing starts.
- The major process scheduling algorithms based on batch system scheduling are:
 - First Come First Serve (FCFS)
 - Shortest Job First (SJF)
 - Shortest Remaining Time Next (SRTN)

Interactive System Scheduling

The process scheduling algorithms based on Interactive system scheduling are:

- Round Robin Scheduling
- Priority Scheduling
- Multiple queues (Multilevel Queue Scheduling)
- Multilevel Feedback Queue Scheduling

Real Time System Scheduling

The major process scheduling algorithms based on Real Time scheduling are:

- Priority Fair Share Scheduling
- Guaranteed Scheduling
- Lottery Scheduling
- Highest Response Ratio Next (HRN)

First Come First Serve (FCFS)

- In FCFS, the processes are scheduled in the order they are received.
- FCFS provides an efficient, simple and error free process scheduling algorithm that saves valuable CPU resources.
- It uses non-preemptive scheduling. Once the process has the CPU, it runs to completion.
- It can be easily implemented by managing a simple queue or by sorting according to the time the process was received.
- It is Fair to all processes.
- **Problems:**
 - Doesn't guarantee a good response time.
 - It has large average waiting time.

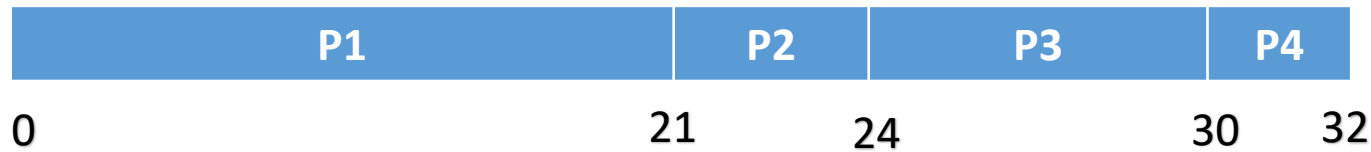
FCFS (Contd..)

- **Example 1:** Consider the processes P1, P2, P3 and P4 given in the table below, arrives for execution in the order, with zero arrival time and given burst time. Find the average waiting time and average turnaround time using FCFS scheduling algorithm.

Process	Burst Time
P1	21
P2	3
P3	6
P4	2

FCFS (Contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	21-0=21	21-21=0
P2	24-0=24	24-3=21
P3	30-0=30	30-6=24
P4	32-0=32	32-2=30

Average Turnaround time = $(21+24+30+32)/4 = 107/4 = 26.75$

Average Waiting time = $(0+21+24+30)/4 = 75/4 = 18.75$

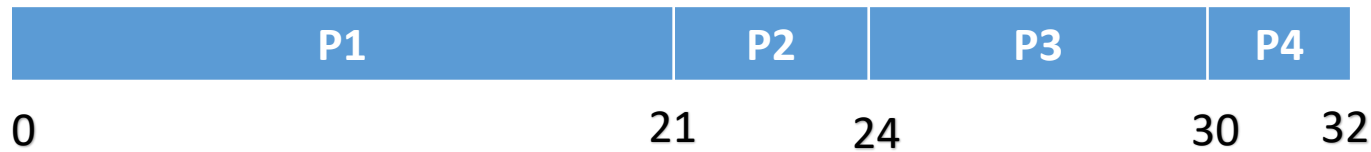
FCFS (Contd..)

- **Example 2:** Consider the processes P1, P2, P3 and P4 given in the table below, arrival for execution and burst time. Find the average waiting time and average turnaround time using FCFS scheduling algorithm.

Process	Arrival Time	Burst Time
P1	0 ms	21 ms
P2	2 ms	3 ms
P3	2 ms	6 ms
P4	3 ms	2 ms

FCFS (Contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	21-0=21	21-21=0
P2	24-2=22	22-3=19
P3	30-2=28	28-6=22
P4	32-3=29	29-2=27

Average Turnaround time = $(21+22+28+29)/4 = 100/4 = 25$ ms

Average Waiting time = $(0+19+22+27)/4 = 68/4 = 17$ ms

Shortest Job First (SJF)

- SJF select the process with the shortest expected processing time. In case of the tie, FCFS Scheduling is used.
- It also uses non-preemptive scheduling. Once the process has the CPU, it runs to completion.
- To use SJF, the processing time for a process must be known in advance. As the decision policies are based on the CPU burst time.
- Advantages:
 - Reduces the average waiting time over FCFS.
 - Favors shorts jobs at the costs of long jobs.
- Problems:
 - It may lead to starvation if only short burst time processes are coming in the ready state.
 - Not applicable in timesharing system.

SJF (Contd..)

- **Example 1:** Consider the processes P1, P2, P3 and P4 given in the table below, arrives for execution in the order, with zero arrival time and given burst time. Find the average waiting time and average turnaround time using SJF scheduling algorithm.

Process	Burst Time
P1	21
P2	3
P3	6
P4	2

SJF (Contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P4	2-0=2	2-2=0
P2	5-0=5	5-3=2
P3	11-0=11	11-6=5
P1	32-0=32	32-21=11

Average Turnaround time = $(2+5+11+32)/4 = 50/4 = 12.5$

Average Waiting time = $(0+2+5+11)/4 = 18/4 = 4.50$

SJF (Contd..)

- **Example 2:** Consider the processes P1, P2, P3 and P4 given in the table below, arrival for execution and burst time. Find the average waiting time and average turnaround time using SJF scheduling algorithm.

Process	Arrival Time	Burst Time
P1	0 ms	21 ms
P2	2 ms	3 ms
P3	2 ms	6 ms
P4	3 ms	2 ms

SJF (Contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	21-0=21	21-21=0
P4	23-3=20	20-2=18
P2	26-2=24	24-3=21
P3	32-2=30	30-6=24

Average Turnaround time = $(21+20+24+30)/4 = 95/4 = 23.75$ ms

Average Waiting time = $(0+18+21+24)/4 = 63/4 = 15.75$ ms

Shortest Remaining Time Next

- It is a preemptive version of SJF scheduling.
- It selects the process with the smallest amount of time remaining required for completion.
- It is advantageous because short processes are handled very quickly.
- The system also requires very little overhead since it only makes a decision when a process completes or a new process is added, and when a new process is added the algorithm only needs to compare the currently executing process with the new process, ignoring all other processes currently waiting to execute.

Shortest Remaining Time Next (contd..)

Merits

- Low average waiting time than SJF.
- Useful in timesharing.

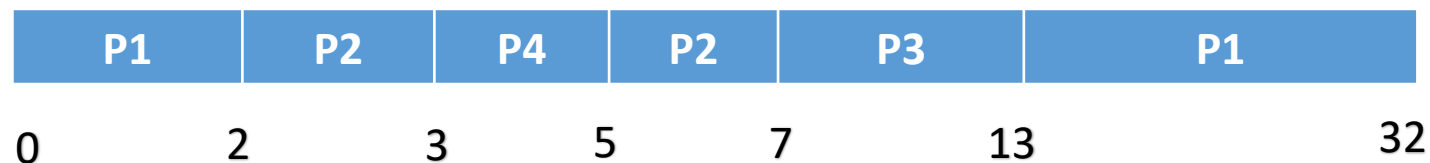
Demerits

- Very high overhead than SJF.
- Requires additional computation.
- Favors short jobs, long jobs can be victims of starvation.

Shortest Remaining Time Next (contd..)

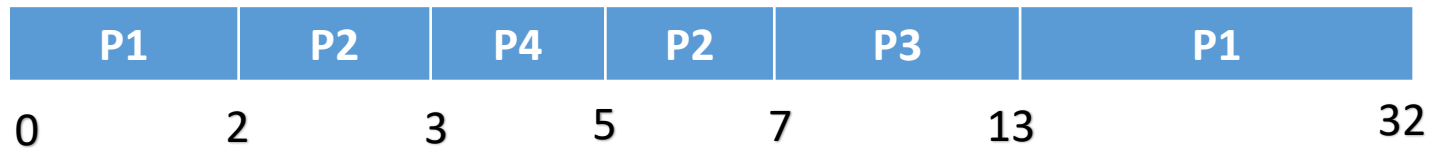
- **Example 1:** Consider the processes P1, P2, P3 and P4 given in the table below, arrival for execution and burst time. Find the average waiting time and average turnaround time using SRTN scheduling algorithm.

Process	Arrival Time	Burst Time
P1	0 ms	21 ms
P2	2 ms	3 ms
P3	2 ms	6 ms
P4	3 ms	2 ms



Shortest Remaining Time Next (contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	$32-0=32$	$32-21=11$
P2	$7-2=5$	$5-3=2$
P4	$5-3=2$	$2-2=0$
P3	$13-2=11$	$11-6=5$

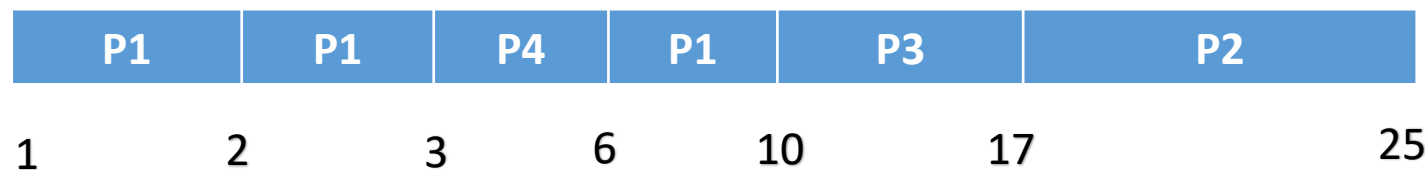
Average Turnaround time = $(32+5+2+11)/4 = 50/4 = 12.5$ ms

Average Waiting time = $(11+2+0+5)/4 = 18/4 = 4.5$ ms

Shortest Remaining Time Next (contd..)

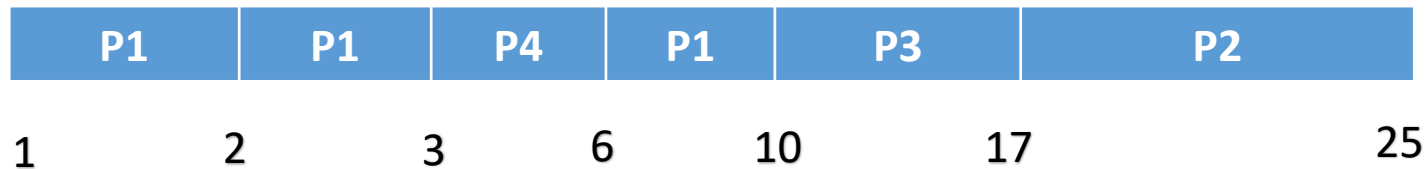
- **Example 2:** Consider the processes P1, P2, P3 and P4 given in the table below, arrival for execution and burst time. Find the average waiting time and average turnaround time using SRTN scheduling algorithm.

Process	Arrival Time	Burst Time
P1	1 ms	6 ms
P2	1 ms	8 ms
P3	2 ms	7 ms
P4	3 ms	3 ms



Shortest Remaining Time Next (contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	10-1=9	9-6=3
P2	25-1=24	24-8=16
P3	17-2=15	15-7=8
P4	6-3=3	3-3=0

Average Turnaround time = $(9+24+15+3)/4 = 51/4 = 12.75$ ms

Average Waiting time = $(3+16+8+0)/4 = 27/4 = 6.75$ ms

Round Robin (RR) Scheduling

- This algorithm allocates the CPU for the specific time period called time slice or quantum.
- If the process completes its execution within the defined quantum, then it is removed from the queue otherwise it has to wait for another time slice and is placed at the back of the ready queue.
- If the quantum is very large, each process is given as much time as required for completion. It will be similar to FCFS.
- If quantum is small, system will be busy at just switching from one process to another process, the overhead of context switching causes the system efficiency degrading.

Round Robin (RR) Scheduling

Advantages:

- Fair allocation of CPU across the process.
- Used in timesharing system.
- Low average waiting time when process lengths vary widely.

Disadvantages:

- Poor average waiting time when process lengths are identical.
- We have to perform a lot of context switching here, which will keep the CPU idle.

Round Robin (RR) Scheduling (contd..)

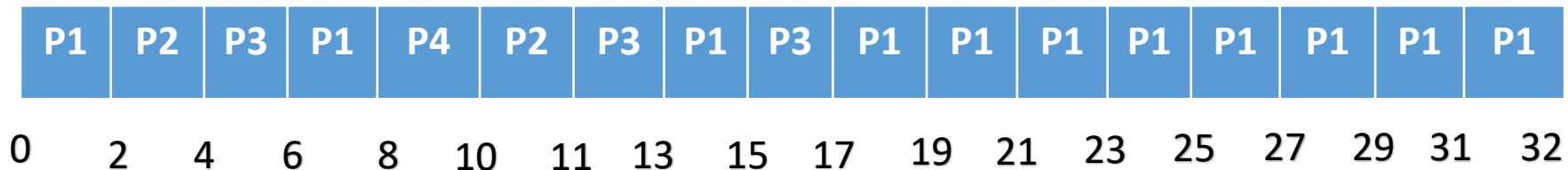
- **Example 1:** Consider the processes P1, P2, P3 and P4 given in the table below, arrival for execution and burst time. Find the average waiting time and average turnaround time using RR scheduling algorithm (Quantum time=2ms).

Process	Arrival Time	Burst Time
P1	0 ms	21 ms
P2	2 ms	3 ms
P3	2 ms	6 ms
P4	3 ms	2 ms

P1	P2	P3	P1	P4	P2	P3	P1	P3	P1	P1	P1	P1	P1	P1	P1	P1	
0	2	4	6	8	10	11	13	15	17	19	21	23	25	27	29	31	32

Round Robin (RR) Scheduling (contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	$32-0=32$	$32-21=11$
P2	$11-2=9$	$9-3=6$
P4	$10-3=7$	$7-2=5$
P3	$17-2=15$	$15-6=9$

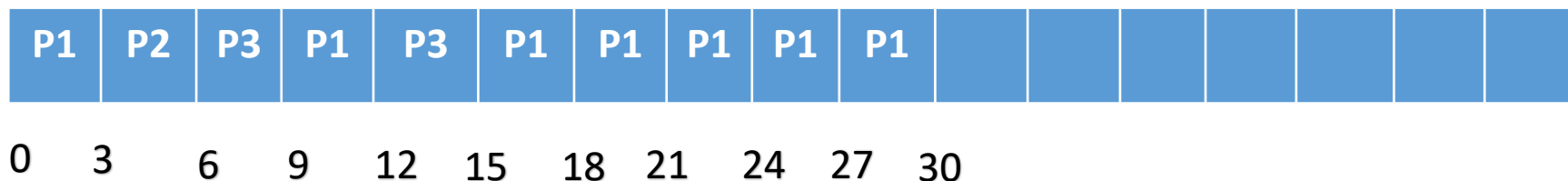
Average Turnaround time = $(32+9+7+15)/4 = 63/4 = 15.75$ ms

Average Waiting time = $(11+6+5+9)/4 = 31/4 = 7.75$ ms

Round Robin (RR) Scheduling (contd..)

- **Example 2:** Consider the processes P1, P2 and P3 given in the table below with arrival time 0 for execution and burst time. Find the average waiting time and average turnaround time using RR scheduling algorithm (Quantum time=3ms).

Process	Burst Time
P1	21 ms
P2	3 ms
P3	6 ms



Priority Scheduling

- In this algorithm, the priority is assigned to all the processes and the process with highest priority is executed first.
- Priority assignment of processes is done on the basis of internal factor such as CPU and memory requirements or external factor such as user's choice. It is just used to identify which process is having a higher priority and which process is having a lower priority.
- It is a non-preemptive scheduling.

Priority Scheduling (contd..)

Advantages

- Higher priority processes like system processes are executed first.

Disadvantages

- It can lead to starvation if only higher priority process comes into the ready state. Low priority processes may never execute.
- If the priorities of two processes are the same, then we have to use some other scheduling algorithm.

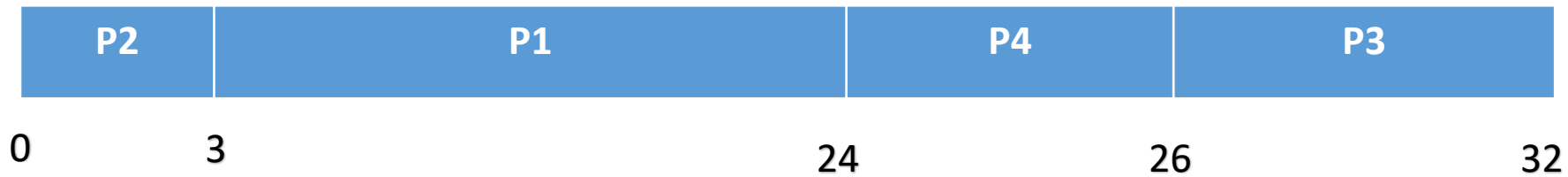
Priority Scheduling (contd..)

- Example 1: Consider the processes P1, P2, P3 and P4 given in the table below, arrives for execution in the same order, with arrival time 0, and given burst time. Find the average waiting time and average turnaround time using the Priority Scheduling algorithm.

Process	Burst Time	Priority
P1	21	2
P2	3	1
P3	6	4
P4	2	3

Priority Scheduling (contd..)

Gantt Chart:



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P2	3-0=3	3-3=0
P1	24-0=24	24-21=3
P4	26-0=26	26-2=24
P3	32-0=32	32-6=26

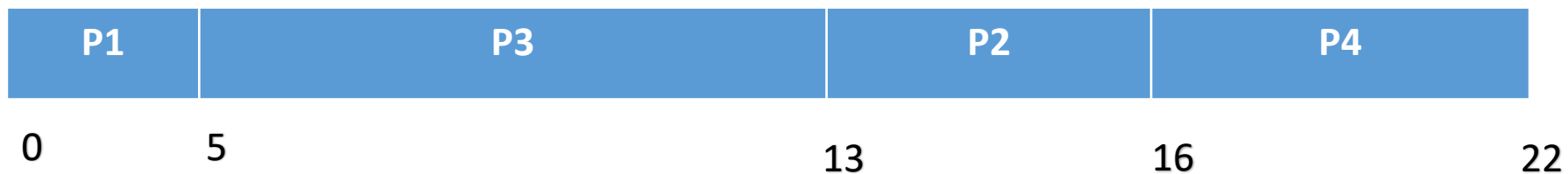
Average Turnaround time = $(3+24+26+32)/4 = 85/4 = 21.25$ ms

Average Waiting time = $(0+3+24+26)/4 = 53/4 = 13.25$ ms

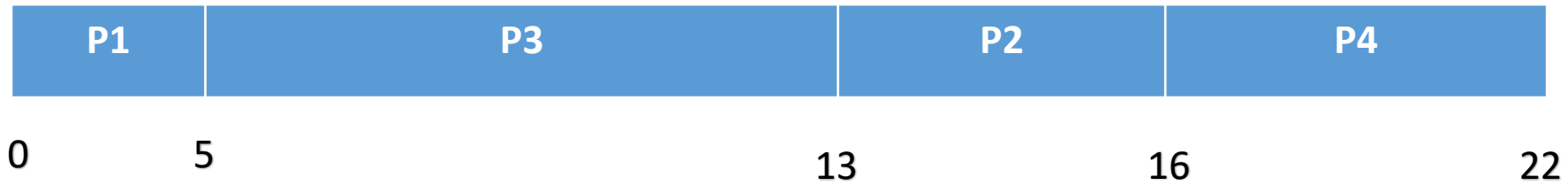
Priority Scheduling (contd..)

- Example 2: Consider the processes P1, P2, P3 and P4 given in the table below, with arrival time and given burst time. Find the average waiting time and average turnaround time using the Priority Scheduling algorithm.

Process	Arrival Time	Burst Time	Priority
P1	0 ms	5 ms	1
P2	1 ms	3 ms	2
P3	2 ms	8 ms	1
P4	3 ms	6 ms	3



Priority Scheduling (contd..)



Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	5-0=5	5-5=0
P3	13-2=11	11-8=3
P2	16-1=15	15-3=12
P4	22-3=19	19-6=13

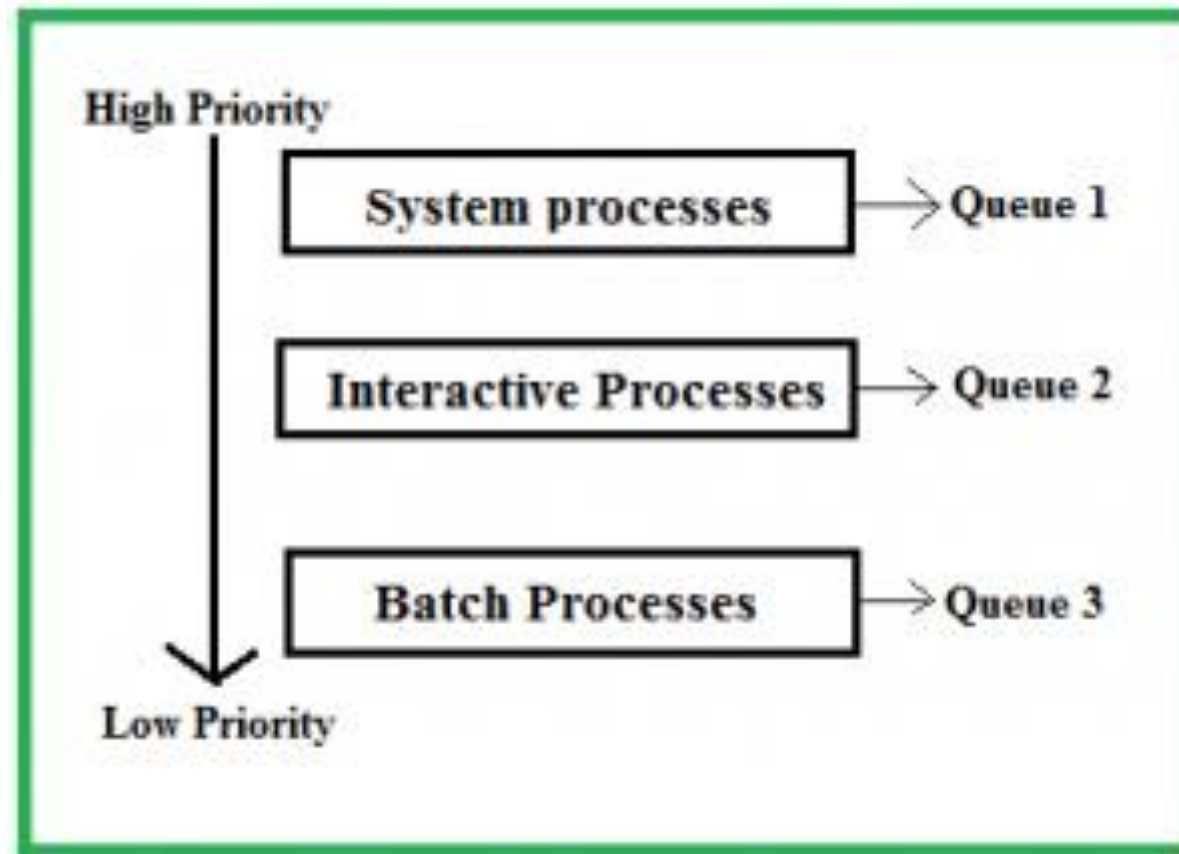
Average Turnaround time = $(5+11+15+19)/4 = 50/4 = 12.5$ ms

Average Waiting time = $(0+3+12+13)/4 = 28/4 = 7$ ms

Multiple Queues (Multilevel Queue Scheduling)

- The processes in the ready queue can be divided into different classes where each class has its own scheduling needs. For example, a common division is a foreground (interactive) process and background (batch) processes.
- These two classes have different scheduling needs. For this kind of situation, Multiple Queue Scheduling is used.
- Ready Queue is divided into separate queues for each class of processes. For example, let us take three different types of process: System process, Interactive process, and Batch Process. All three processes have their own queue and its own scheduling algorithm.

Multiple Queues (Multilevel Queue Scheduling)



Multiple Queues (Multilevel Queue Scheduling)

- Example 1: Consider the table below of four processes under multiple queue scheduling. Queue number denotes the queue of the process.

Process	Arrival Time	Burst Time	Queue Number
P1	0 ms	5 ms	1
P2	0 ms	3 ms	2
P3	0 ms	8 ms	2
P4	0 ms	6 ms	1

Here, there are two queues: Queue 1 and Queue 2. Queue 1 is having higher priority and is using the FCFS. Queue 2 is using RR (Quantum=2).

Use multiple queue and calculate average waiting time and turn around time.

Multiple Queues (Multilevel Queue Scheduling)

P1		P4			P2	P3		P2	P3	P3	P3	
0	5	11	13	15	16	18	20	22				

Calculation:

Process	Turnaround time (Completion time-arrival time)	Waiting time (Turnaround time-burst time)
P1	5-0=5	5-5=0
P4	11-0=11	11-6=5
P2	16-0=16	16-3=13
P3	22-0=22	22-8=14

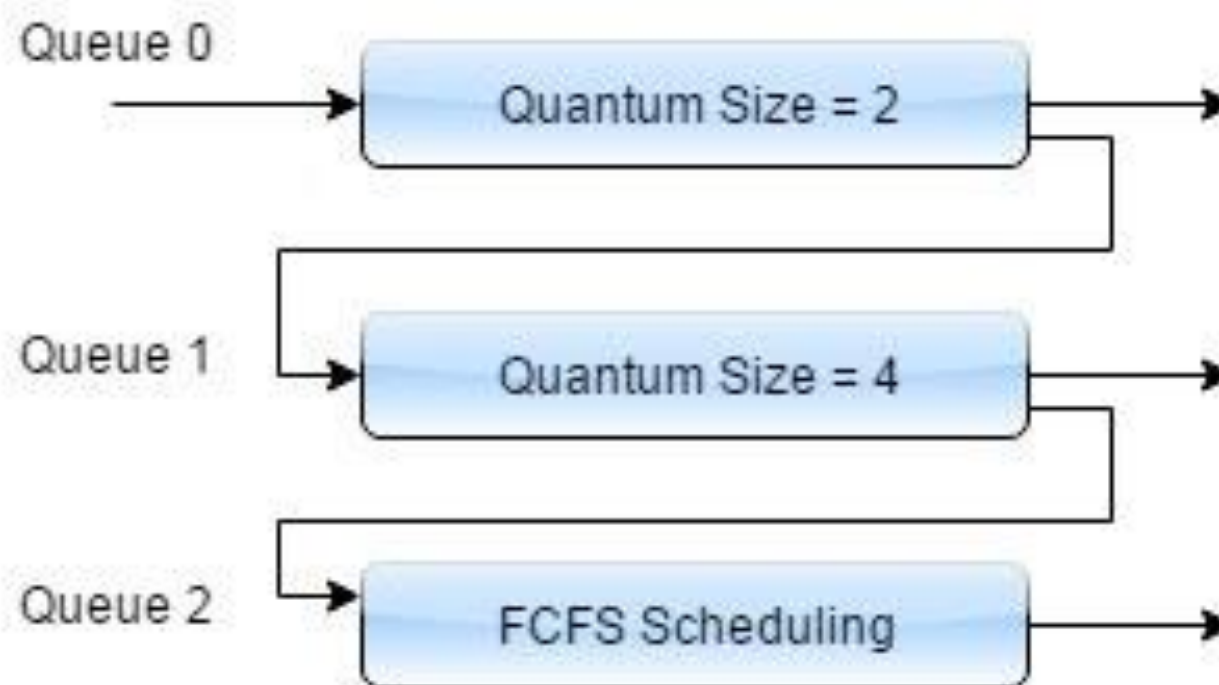
Average Turnaround time = $(5+11+16+22)/4 = 54/4 = 13.5$ ms

Average Waiting time = $(0+5+13+14)/4 = 32/4 = 8$ ms

Multilevel Feedback Queue Scheduling

- It is the modification of multiple queue.
- It allows the process to move in between queue.
- The idea is to separate processes according to the characteristics of their CPU bursts.
- If a process uses too much CPU time, it is moved to a lower-priority queue. Similarly, a process that waits too long in a lower-priority queue may be moved to higher-priority queue.

Multilevel Feedback Queue Scheduling



Real Time Scheduling

- It is defined for real time systems.
- It is composed of the scheduler, clock and the processing hardware elements.
- It includes:
 - Priority Fair Share Scheduling
 - Guaranteed Scheduling
 - Lottery Scheduling
 - Highest Response Ratio Next (HRN)

Priority Fair Share Scheduling

- It is used in OS in which CPU usage is equally distributed among system users or groups, as opposed to equal distribution among processes.
- If four users (A, B, C, D) are concurrently executing one process each, the scheduler will logically divide the available CPU cycles such that each users gets 25% of the whole ($100\%/4=25\%$).
- One common method of logically implementing the fair-share scheduling strategy is to recursively apply the round robin scheduling strategy at each level of abstraction (process, user, group).

Guaranteed Scheduling

- It is a scheduling algorithm used in multitasking OS that guarantees fairness by monitoring the amount of CPU time spent by each user and allocating resources accordingly.
- It makes real promises to the users about the performance.
- If there are n users logged in, each user will receive $1/n$ of the CPU power. Similarly, on a single user system with n processes running, all things being equal, each one should get $1/n$ of the CPU cycles.
- To make good on this promise, the system must keep track of how much CPU each process has had since its creation. It compute the ratio of actual CPU time consumed to the CPU time entitled.
- The ratio of 0.5 means that the process only had 50% of what it should have had.

Lottery Scheduling

- Processes are scheduled in a random manner.
- It can be preemptive or non-preemptive.
- In this scheduling, each process is given at least one lottery ticket and scheduler picks a random ticket and process having that ticket is executed.
- The process having higher number of tickets will have higher chance of execution.

Highest Response Ratio Next (HRN)

- It is a non-preemptive scheduling algorithm.
- In Shortest Job First (SJF) scheduling, priority is given to shortest job, which may sometimes result in indefinite blocking of longer job. HRN is used to correct this disadvantage of SJF.
- To determine the priority for selecting the process, not only the burst time but the waiting time is also considered.
- It uses dynamic priorities instead of fixed priorities. Dynamic priorities in HRN is calculated as: $\text{priority} = (\text{waiting time} + \text{burst time}) / \text{burst time}$.
- So, shorter jobs get preference over longer processes. Similarly, longer jobs that have been waiting for long period are also given favorable treatment.

Highest Response Ratio Next (HRN)

Advantages

- Its performance is better than SJF Scheduling
- It limits the waiting time of longer jobs and also supports shorter jobs.

Disadvantages

- It can't be implemented practically because the burst time of all the processes cannot be known in advance.